
INTRODUÇÃO A ARQUITETURAS PARALELAS

Computação Paralela — Unidade 1

06 de Agosto, 2026

ROTEIRO

- 1 | Histórico e evolução das arquiteturas
- 2 | Classes de arquiteturas paralelas e distribuídas
- 3 | Sistemas concorrentes, paralelos e distribuídos
- 4 | Métricas para avaliação

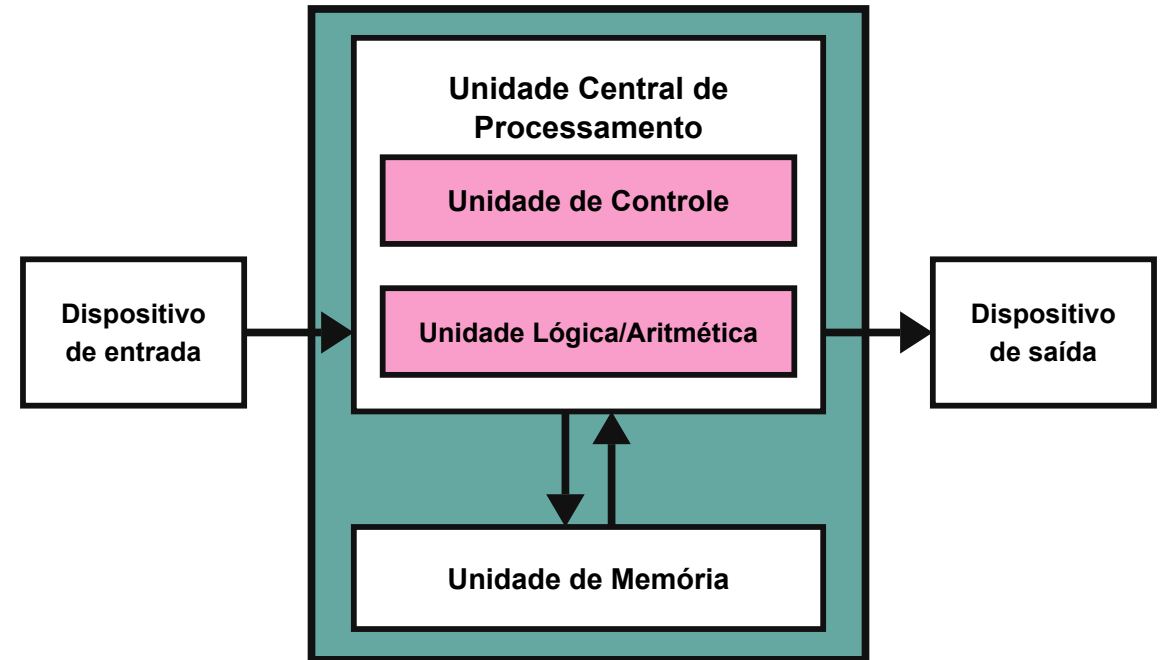
ARQUITETURA DE VON NEUMANN

Instruções e dados na mesma memória;
execução sequencial.

CICLO DE INSTRUÇÃO

1. **Fetch** — busca
2. **Decode** — decodificação
3. **Execute** — execução
4. **Write** — escrita

Uma única unidade ativa por vez. A evolução busca *superar* essa limitação.



EVOLUÇÃO DAS ARQUITETURAS

Objetivo: acelerar o processamento dos dados pela CPU.

LIBERAR A CPU

- Controle do barramento
- Tratamento de E/S
- DMA, multiprogramação

ACCELERAR ALIMENTAÇÃO

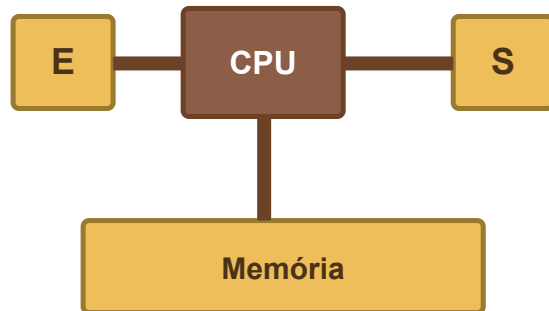
- Hierarquia de memória
- Caches L1, L2, L3
- Localidade temporal/espacial

SOBREPOR CICLOS

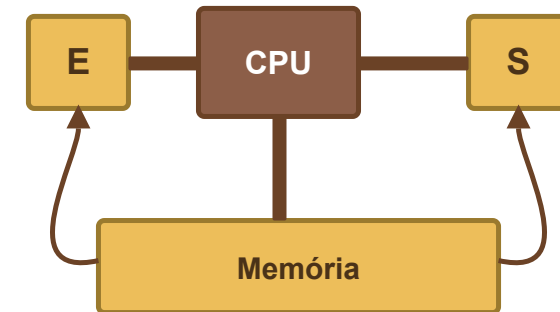
- Pipeline de instrução
- Múltiplas threads
- Múltiplos cores

DIRECT MEMORY ACCESS (DMA)

Problema: na arquitetura tradicional, a CPU intermedia **toda** transferência entre E/S e memória — fica ocupada durante todo o processo.



Solução — DMA: unidades de E/S autônomas transferem **blocos** diretamente para a memória. A CPU fica livre.

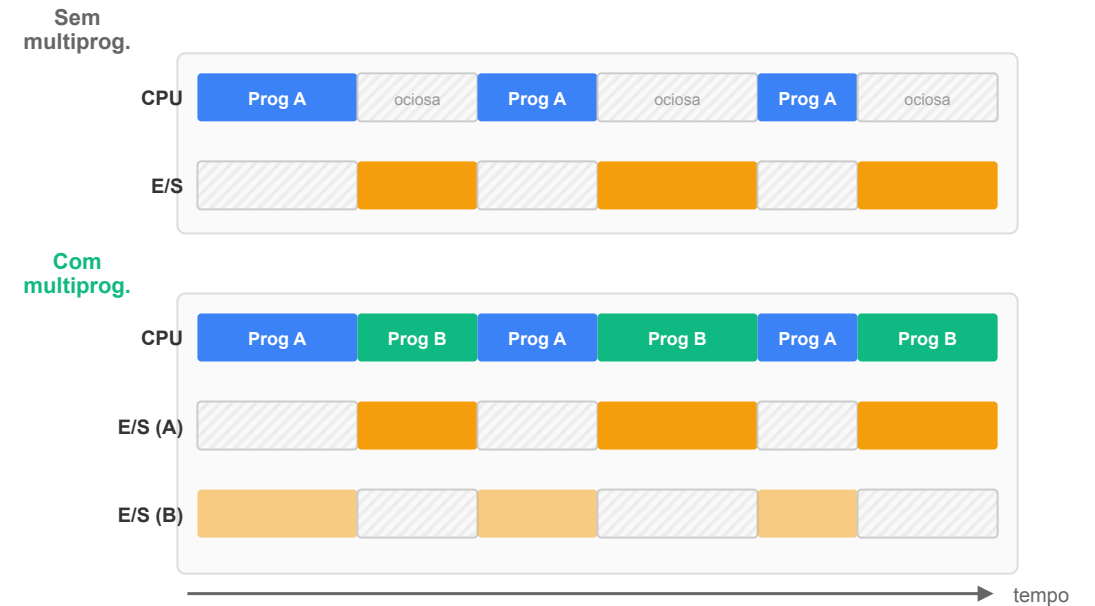


MULTIPROGRAMAÇÃO

Vários processos compartilham a CPU (*time-sharing*); enquanto um espera E/S, outro executa.

COMO FUNCIONA

- SO alterna processos na CPU
- Quando um processo bloqueia (E/S), outro ocupa a CPU
- Aumenta a **utilização** da CPU sem paralelismo real



DMA + MULTIPROGRAMAÇÃO

DMA

Libera a CPU durante transferências de E/S

Controlador transfere blocos de dados diretamente entre E/S e memória

MULTIPROGRAMAÇÃO

Ocupa a CPU com outro processo enquanto um espera E/S

SO alterna processos — a CPU nunca fica ociosa

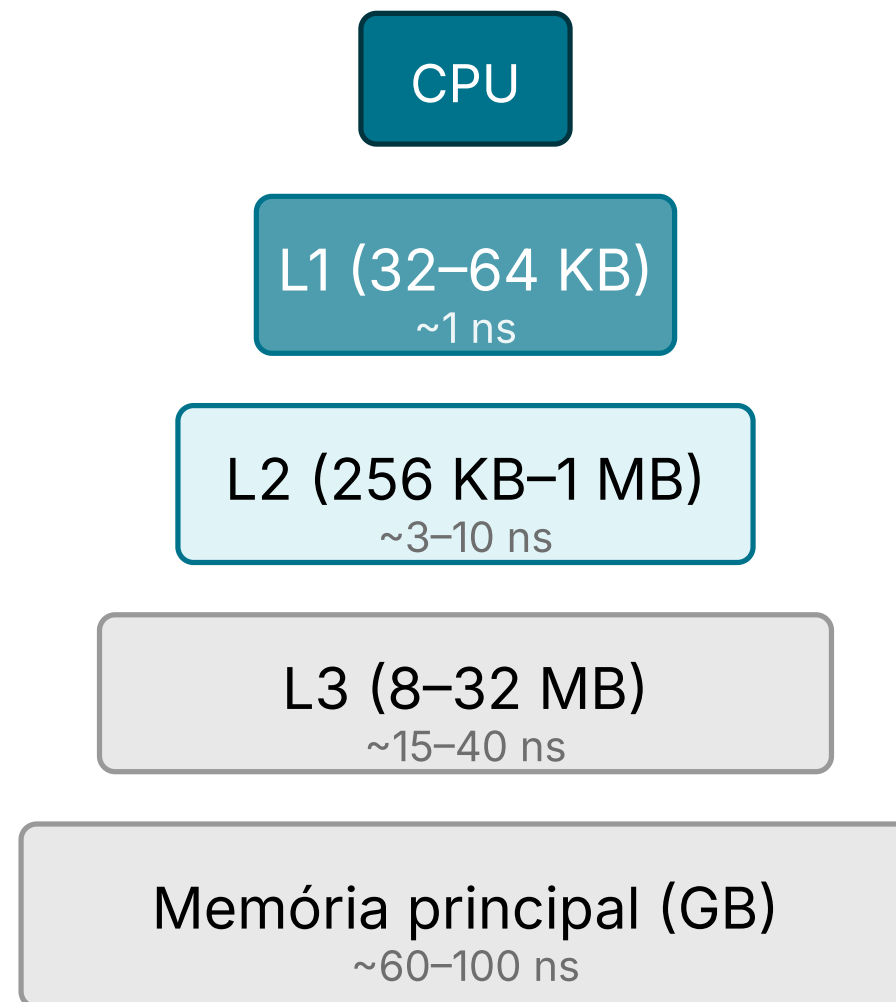
Juntas, DMA e multiprogramação permitem **concorrência** em um único processador — múltiplos processos progridem simultaneamente, sem precisar de múltiplos cores.

HIERARQUIA DE MEMÓRIA

Caches (L1, L2, L3) aceleram a alimentação da CPU com dados e instruções.

- **Localidade temporal:** dados usados recentemente tendem a ser usados de novo
- **Localidade espacial:** dados próximos em endereço são acessados juntos

Quanto mais perto da CPU, menor a latência e o tamanho.



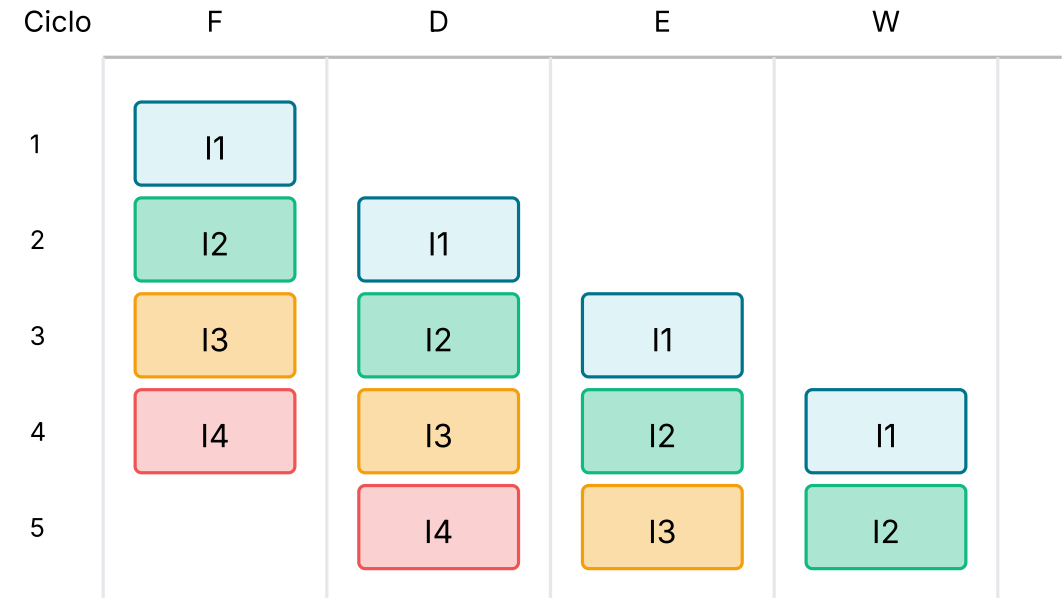
PIPELINES

DE INSTRUÇÃO (SUPERESCALAR)

Instruções passam por estágios (Fetch, Decode, Execute, Write); várias instruções em diferentes estágios ao mesmo tempo.

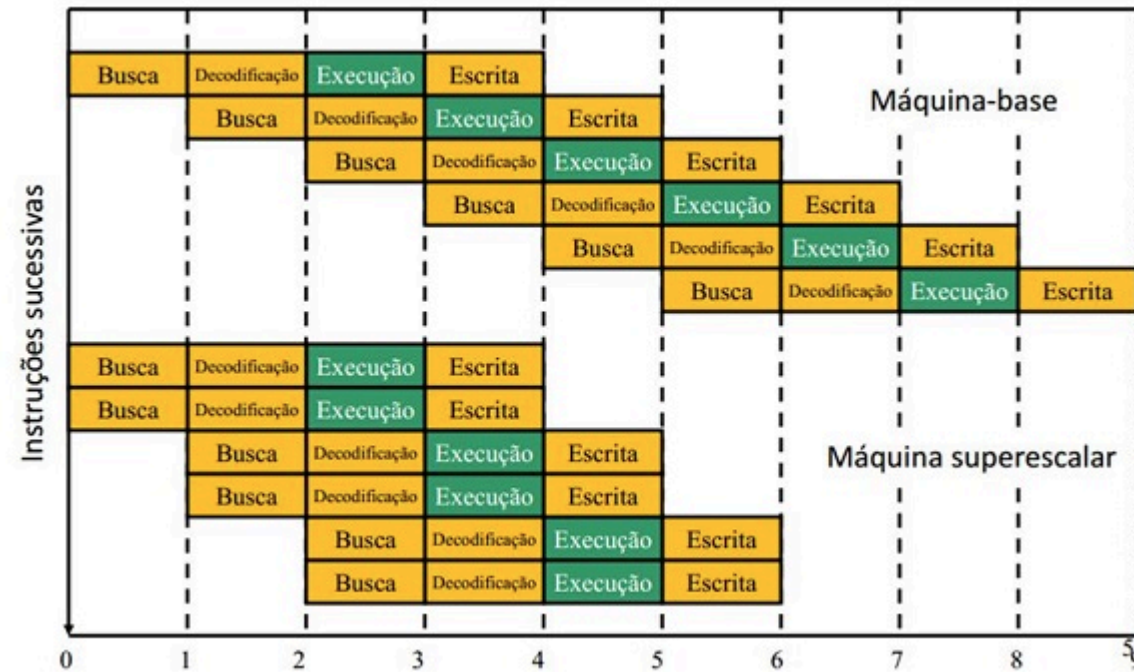
DE DADOS (VETORIAL)

Máquinas vetoriais: mesma operação aplicada a vetores de dados (SIMD).



Após “encher” o pipeline, tende a completar ~1 instrução por ciclo (throughput).

PIPELINE SUPERESCALAR

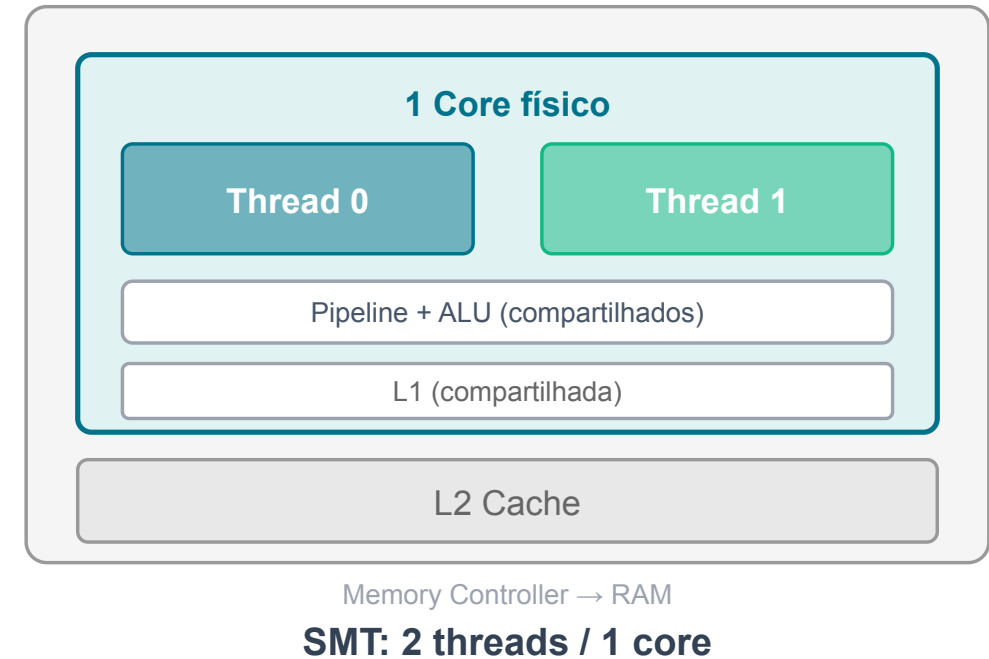


Máquina-base: uma instrução por ciclo — **Superescalar:** múltiplas instruções por ciclo

HYPER-THREADING (SMT)

Tecnologia Intel (2002): melhor aproveitar pipelines superescalares com **Simultaneous Multithreading (SMT)**.

- Um núcleo físico expõe *duas* threads lógicas
- Quando uma thread espera (cache miss, dependência), a outra usa os recursos ociosos
- Ganhos de até ~30% em workloads adequados



HYPER-THREADING — COMO FUNCIONA

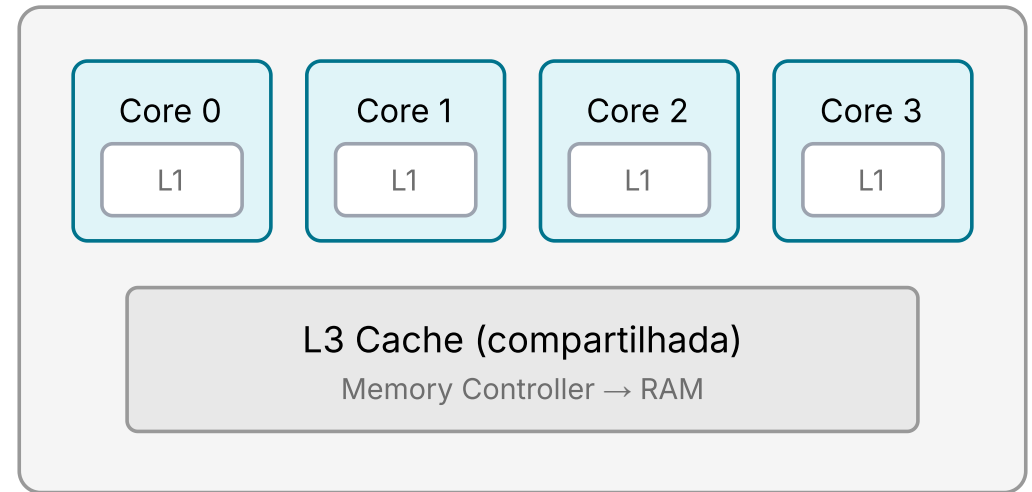
<https://www.youtube.com/watch?v=CNWtuEUHESA>

DUAL-CORE E MULTICORE

Múltiplos núcleos de processamento no mesmo chip. Alternativa ao aumento de frequência (limites térmicos, power wall).

- **Dual-core:** Intel Pentium D (2005), AMD Athlon 64 X2
- **Multicore:** Intel Core 2 Quad (2007); foco em eficiência energética

Melhor desempenho com menor consumo: mais trabalho por watt.



Ex.: Intel Core i7-3770K (4 cores, 8 threads com HT)

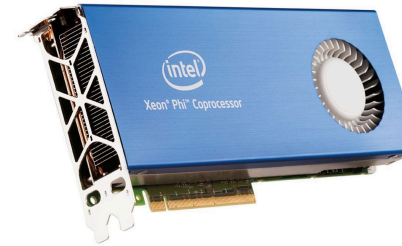
DUAL-CORE — COMO FUNCIONA

<https://www.youtube.com/watch?v=0Fqc8k4uzLU>

MANYCORE E GPUS

Centenas ou milhares de núcleos no mesmo chip; co-processadores especializados (GPGPU/NPU).

- **GPU:** Graphics Processing Unit, inicialmente gráficos; hoje uso geral (IA, simulação, HPC)
- **NPU:** Neural Processing Unit, especializada em tarefas de IA
- **Intel Xeon Phi:** manycore x86 em placa PCIe
- **NVIDIA Titan V / Tesla:** milhares de cores SIMT



Intel Xeon Phi



NVIDIA Titan V

ARQUITETURA DE GPU — NVIDIA TESLA V100

- **GPC** — Graphics Processing Cluster
- **TPC** — Texture Processing Cluster
- **SM** — Streaming Multiprocessor

Hierarquia: cada GPC contém vários TPCs; cada TPC contém SMs com dezenas de cores CUDA.

CPU + GPU em conjunto: **computação heterogênea** — CPU orquestra, GPU acelera cargas paralelas de dados.



LEI DE MOORE

O número de transistores em um chip dobra aproximadamente a cada **18–24 meses**.

O QUE ACONTECEU

- Transistores continuaram dobrando
- Mas a frequência **estagnou** por volta de 2005
- Limites de energia e calor (*fim do Dennard scaling*)

PARA ONDE FORAM OS TRANSISTORES

- **Mais cores** — paralelismo no chip
- **Mais cache** — L3 cada vez maior
- **Unidades especializadas** — GPUs, NPUs, aceleradores

Consequência: ganho de desempenho exige **paralelismo explícito** no software — não basta esperar o hardware ficar mais rápido.

Our World
in Data

Lei de Moore: número de transistores dos chips irá dobrar a cada 18 meses.

Transistor count

Year of introduction

Key data points labeled in the chart:

- Intel 4004 (1971)
- Intel 8008 (1972)
- Intel 8080 (1974)
- Intel 8085 (1976)
- Intel 8086 (1978)
- Intel 8088 (1982)
- Intel 80286 (1985)
- Intel 80386 (1985)
- Intel 80486 (1989)
- Intel Pentium (1993)
- Intel Pentium Pro (1995)
- Intel Pentium 4 (2000)
- Intel Core 2 Duo (2006)
- Intel Core i7 (2008)
- Intel Core i3 (2010)
- Intel Core i5 (2012)
- Intel Core i7 (2014)
- Intel Core i9 (2017)
- ARM 1 (1985)
- ARM 2 (1986)
- ARM 3 (1987)
- ARM 6 (1991)
- ARM Cortex-A9 (2008)
- ARM Cortex-A53 (2015)
- ARM Cortex-A72 (2016)
- ARM Cortex-X2 (2022)
- Apple A7 (2012)
- Apple A10 (2015)
- Apple A11 (2016)
- Apple A12 (2018)
- Apple A13 (2019)
- Apple A14 (2020)
- Apple A15 (2021)
- Apple A16 (2022)
- Apple M1 (2020)
- Apple M2 (2022)
- Apple M3 (2023)
- Apple M4 (2024)
- Apple M5 (2025)
- Apple M6 (2026)
- Apple M7 (2027)
- Apple M8 (2028)
- Apple M9 (2029)
- Apple M10 (2030)
- Apple M11 (2031)
- Apple M12 (2032)
- Apple M13 (2033)
- Apple M14 (2034)
- Apple M15 (2035)
- Apple M16 (2036)
- Apple M17 (2037)
- Apple M18 (2038)
- Apple M19 (2039)
- Apple M20 (2040)
- Apple M21 (2041)
- Apple M22 (2042)
- Apple M23 (2043)
- Apple M24 (2044)
- Apple M25 (2045)
- Apple M26 (2046)
- Apple M27 (2047)
- Apple M28 (2048)
- Apple M29 (2049)
- Apple M30 (2050)
- Apple M31 (2051)
- Apple M32 (2052)
- Apple M33 (2053)
- Apple M34 (2054)
- Apple M35 (2055)
- Apple M36 (2056)
- Apple M37 (2057)
- Apple M38 (2058)
- Apple M39 (2059)
- Apple M40 (2060)
- Apple M41 (2061)
- Apple M42 (2062)
- Apple M43 (2063)
- Apple M44 (2064)
- Apple M45 (2065)
- Apple M46 (2066)
- Apple M47 (2067)
- Apple M48 (2068)
- Apple M49 (2069)
- Apple M50 (2070)
- Apple M51 (2071)
- Apple M52 (2072)
- Apple M53 (2073)
- Apple M54 (2074)
- Apple M55 (2075)
- Apple M56 (2076)
- Apple M57 (2077)
- Apple M58 (2078)
- Apple M59 (2079)
- Apple M60 (2080)
- Apple M61 (2081)
- Apple M62 (2082)
- Apple M63 (2083)
- Apple M64 (2084)
- Apple M65 (2085)
- Apple M66 (2086)
- Apple M67 (2087)
- Apple M68 (2088)
- Apple M69 (2089)
- Apple M70 (2090)
- Apple M71 (2091)
- Apple M72 (2092)
- Apple M73 (2093)
- Apple M74 (2094)
- Apple M75 (2095)
- Apple M76 (2096)
- Apple M77 (2097)
- Apple M78 (2098)
- Apple M79 (2099)
- Apple M80 (2100)
- Apple M81 (2101)
- Apple M82 (2102)
- Apple M83 (2103)
- Apple M84 (2104)
- Apple M85 (2105)
- Apple M86 (2106)
- Apple M87 (2107)
- Apple M88 (2108)
- Apple M89 (2109)
- Apple M90 (2110)
- Apple M91 (2111)
- Apple M92 (2112)
- Apple M93 (2113)
- Apple M94 (2114)
- Apple M95 (2115)
- Apple M96 (2116)
- Apple M97 (2117)
- Apple M98 (2118)
- Apple M99 (2119)
- Apple M100 (2120)
- Apple M101 (2121)
- Apple M102 (2122)
- Apple M103 (2123)
- Apple M104 (2124)
- Apple M105 (2125)
- Apple M106 (2126)
- Apple M107 (2127)
- Apple M108 (2128)
- Apple M109 (2129)
- Apple M110 (2130)
- Apple M111 (2131)
- Apple M112 (2132)
- Apple M113 (2133)
- Apple M114 (2134)
- Apple M115 (2135)
- Apple M116 (2136)
- Apple M117 (2137)
- Apple M118 (2138)
- Apple M119 (2139)
- Apple M120 (2140)
- Apple M121 (2141)
- Apple M122 (2142)
- Apple M123 (2143)
- Apple M124 (2144)
- Apple M125 (2145)
- Apple M126 (2146)
- Apple M127 (2147)
- Apple M128 (2148)
- Apple M129 (2149)
- Apple M130 (2150)
- Apple M131 (2151)
- Apple M132 (2152)
- Apple M133 (2153)
- Apple M134 (2154)
- Apple M135 (2155)
- Apple M136 (2156)
- Apple M137 (2157)
- Apple M138 (2158)
- Apple M139 (2159)
- Apple M140 (2160)
- Apple M141 (2161)
- Apple M142 (2162)
- Apple M143 (2163)
- Apple M144 (2164)
- Apple M145 (2165)
- Apple M146 (2166)
- Apple M147 (2167)
- Apple M148 (2168)
- Apple M149 (2169)
- Apple M150 (2170)
- Apple M151 (2171)
- Apple M152 (2172)
- Apple M153 (2173)
- Apple M154 (2174)
- Apple M155 (2175)
- Apple M156 (2176)
- Apple M157 (2177)
- Apple M158 (2178)
- Apple M159 (2179)
- Apple M160 (2180)
- Apple M161 (2181)
- Apple M162 (2182)
- Apple M163 (2183)
- Apple M164 (2184)
- Apple M165 (2185)
- Apple M166 (2186)
- Apple M167 (2187)
- Apple M168 (2188)
- Apple M169 (2189)
- Apple M170 (2190)
- Apple M171 (2191)
- Apple M172 (2192)
- Apple M173 (2193)
- Apple M174 (2194)
- Apple M175 (2195)
- Apple M176 (2196)
- Apple M177 (2197)
- Apple M178 (2198)
- Apple M179 (2199)
- Apple M180 (2200)
- Apple M181 (2201)
- Apple M182 (2202)
- Apple M183 (2203)
- Apple M184 (2204)
- Apple M185 (2205)
- Apple M186 (2206)
- Apple M187 (2207)
- Apple M188 (2208)
- Apple M189 (2209)
- Apple M190 (2210)
- Apple M191 (2211)
- Apple M192 (2212)
- Apple M193 (2213)
- Apple M194 (2214)
- Apple M195 (2215)
- Apple M196 (2216)
- Apple M197 (2217)
- Apple M198 (2218)
- Apple M199 (2219)
- Apple M200 (2220)
- Apple M201 (2221)
- Apple M202 (2222)
- Apple M203 (2223)
- Apple M204 (2224)
- Apple M205 (2225)
- Apple M206 (2226)
- Apple M207 (2227)
- Apple M208 (2228)
- Apple M209 (2229)
- Apple M210 (2230)
- Apple M211 (2231)
- Apple M212 (2232)
- Apple M213 (2233)
- Apple M214 (2234)
- Apple M215 (2235)
- Apple M216 (2236)
- Apple M217 (2237)
- Apple M218 (2238)

Licensed under [CC-BY-SA](#) by the author Max Roser

REPLICAÇÃO DA CPU

O próximo passo: acelerar o processamento construindo arquiteturas com **múltiplas CPUs**. Mas há duas formas fundamentalmente diferentes de fazer isso:

ARQUITETURAS PARALELAS

Múltiplos processadores **no mesmo sistema**, com memória compartilhada ou interconexão rápida

- Vetorial, SMP, NUMA
- Supercomputadores (MPP)

ARQUITETURAS DISTRIBUÍDAS

Múltiplos computadores **independentes**, conectados por rede, cada um com memória própria

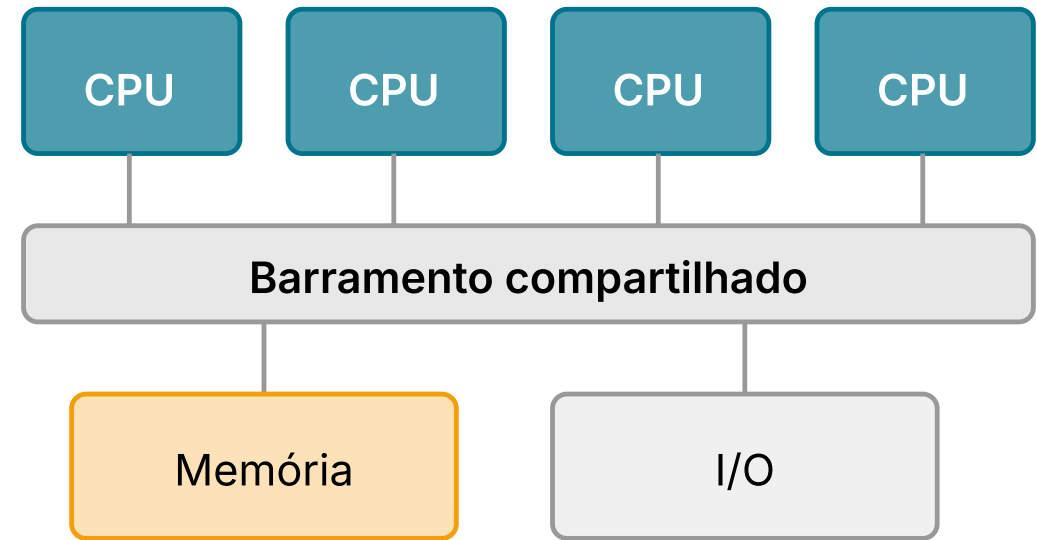
- Clusters (COW)
- Grid computing

SMP = Symmetric Multiprocessing; NUMA = Non-Uniform Memory Access; MPP = Massively Parallel Processing; COW = Cluster of Workstations

SMP — SYMMETRIC MULTIPROCESSING

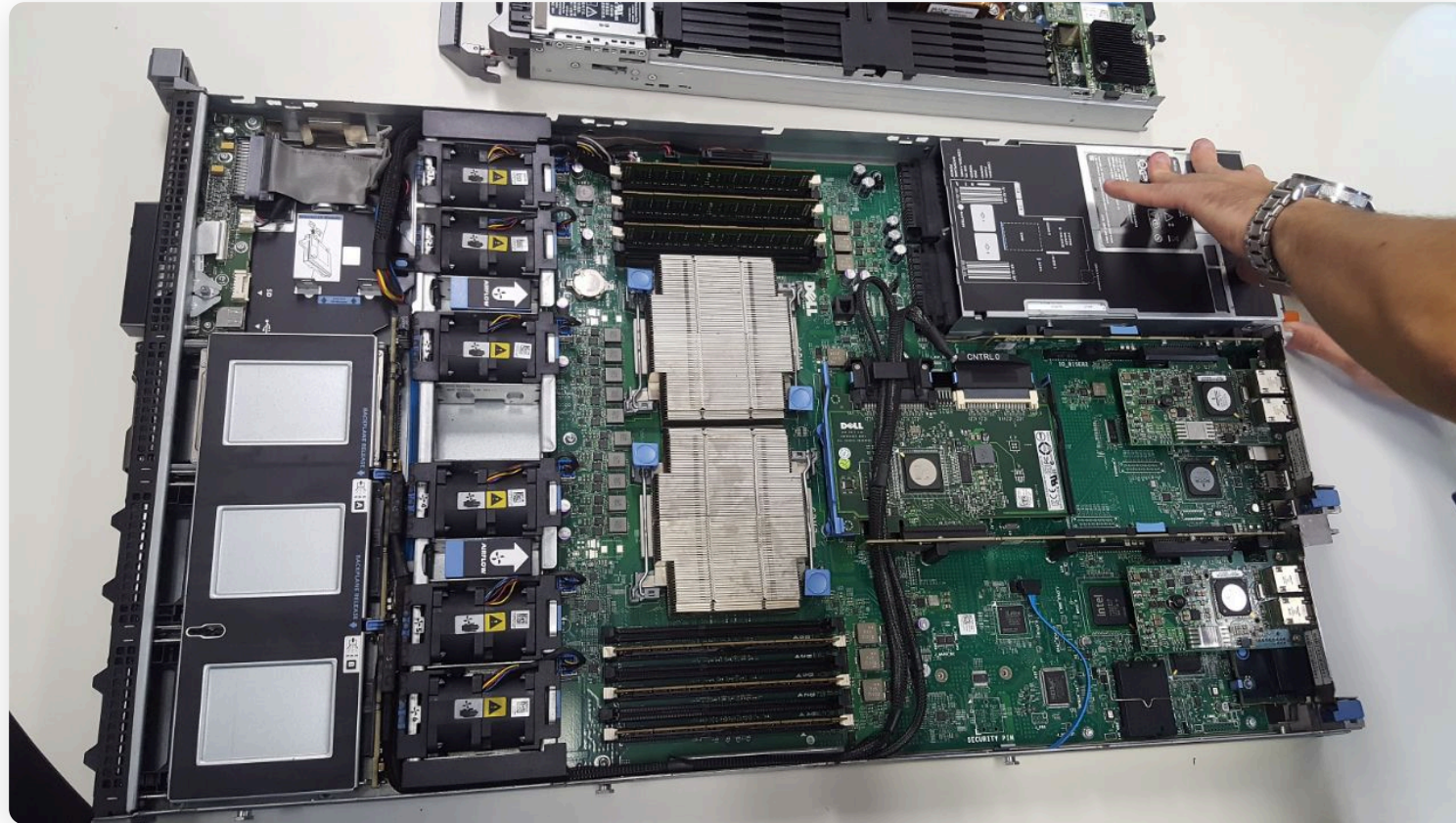
- Computador com múltiplos processadores **idênticos**
- Número limitado de processadores (até **dezenas**)
- Arquitetura de **compartilhamento total** (*shared everything*)
- Executa um **único** sistema operacional

Todos os processadores têm acesso igual à memória, E/S e dispositivos — daí "simétrico".



Shared everything — 1 SO

EXEMPLO DE SERVIDOR SMP

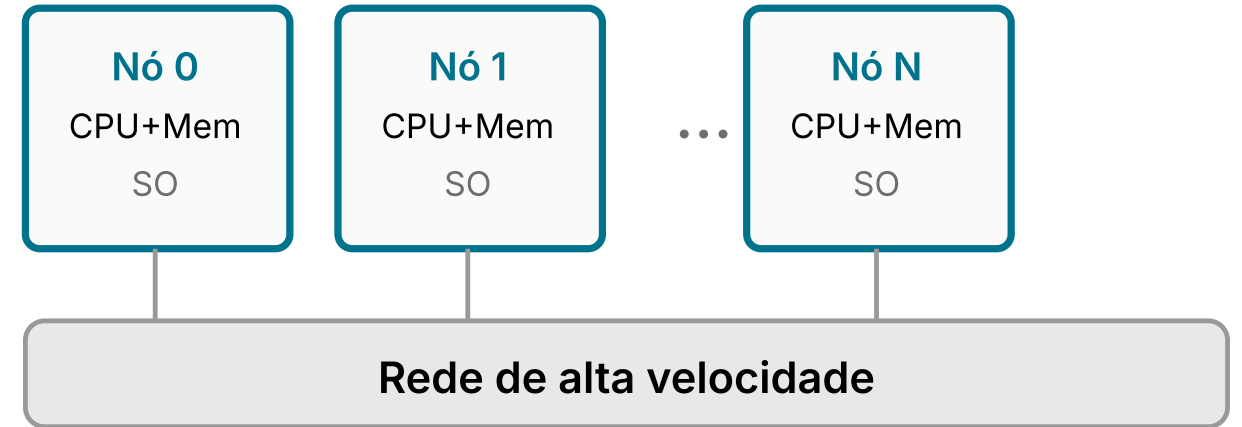


Servidor multiprocessador com dois sockets de CPU e memória compartilhada — arquitetura SMP típica

MPP — MASSIVELY PARALLEL PROCESSING

- Múltiplos computadores interconectados por **rede de alta velocidade**
- Suporta **milhares** de processadores
- Arquitetura **sem compartilhamento** (*shared-nothing*)
- Memória **distribuída** — cada computador tem a sua
- Cada computador executa **seu próprio SO**

Comunicação exclusivamente por **troca de mensagens** via rede dedicada de alta largura de banda e baixa latência.



Shared-nothing — milhares de nós

EXEMPLO DE SUPERCOMPUTADOR MPP

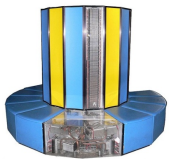


IBM Blue Gene/P — milhares de nós interconectados por rede de alta velocidade

ARQUITETURAS PARALELAS E DISTRIBUÍDAS

Uma escala crescente de poder computacional e complexidade — do compartilhamento de memória à comunicação por rede.

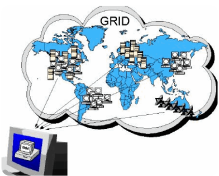
	Memória	Acoplamento	Escala
Vetorial / SMP / NUMA	Compartilhada	Forte	Dezenas
Supercomputadores (MPP)	Distribuída	Forte	Centenas–Milhares
Clusters (COW)	Distribuída	Médio	Centenas–Milhares
Grid	Distribuída	Fraco	Global



Supercomputador Cray



Cluster IBM



Grid Computing

CLASSES DE ARQUITETURAS

Critérios de classificação (Flynn, memória) — e o que cada um implica para o programador

POR QUE ESTUDAR CLASSIFICAÇÕES?

1

Identificar o critério

Por que é importante e quais são as suas implicações?

2

Analisar todas as possibilidades

Inclusive classes não implementadas ou que não deram certo

3

Entender a evolução da área

E como ela pode evoluir no futuro

4

Escolher o modelo de programação certo

A classe da máquina determina a ferramenta (OpenMP, MPI, CUDA) e o que vai limitar o desempenho

CLASSIFICAÇÃO DE FLYNN (1972)

Classificação genérica: fluxo de instruções vs. fluxo de dados — únicos ou múltiplos?

	Fluxo de dados <i>único</i>	Fluxo de dados <i>múltiplo</i>
Fluxo de instruções <i>único</i>	SISD Single Instruction, Single Data Von Neumann, monoprocessador C sequencial	SIMD Single Instruction, Multiple Data GPUs, processadores vetoriais Vetorização, omp simd, AVX, CUDA
Fluxo de instruções <i>múltiplo</i>	MISD Multiple Instruction, Single Data Teórico; raramente implementado	MIMD Multiple Instruction, Multiple Data Multicore, clusters, sistemas distribuídos OpenMP (threads), MPI (processos)

Nota: GPUs modernas usam um modelo chamado SIMT (*Single Instruction, Multiple Threads*) que não é SIMD puro — threads podem divergir, mas os caminhos divergentes são *serializados*, e por isso um `if` que depende da thread custa desempenho. Na classificação de Flynn, são categorizadas como SIMD por ser o comportamento dominante.

MULTIPROCESSADOR VS. MULTICOMPUTADOR

MULTIPROCESSADOR

- Compartilha memória central
- Vários processadores em um sistema
- Comunica por memória (variáveis compartilhadas)

SMP, NUMA

OpenMP — U3

MULTICOMPUTADOR

- Não compartilha memória
- Interligação de arquiteturas tradicionais
- Cada nó com memória local
- Comunica por troca de mensagens

Clusters, COW, MPP

MPI — U4

SMP = Symmetric Multiprocessing; COW = Cluster of Workstations; MPP = Massively Parallel Processor

Essa distinção determina o **modelo de programação**: memória compartilhada (threads e sincronização — **OpenMP**) vs. troca de mensagens (send/receive explícitos — **MPI**).

CLASSIFICAÇÃO POR ESPAÇO DE ENDEREÇAMENTO

MEMÓRIA COMPARTILHADA

- Único espaço de endereçamento
- Comunicação via `load` e `store`
- Variáveis compartilhadas
- Exemplo: SMP, NUMA

Na prática: OpenMP, pthreads. Comunicação *implícita* — não aparece no código, e o custo também não. Exige sincronização e abre espaço para condições de corrida.

MEMÓRIA PRIVADA

- Múltiplos espaços de endereçamento
- Cada processador com memória distinta
- Comunicação via `send` e `receive`
- Exemplo: Clusters, MPP

Na prática: MPI. Comunicação *explícita* — visível no código e mensurável. Não há condições de corrida sobre dados, mas há custo de cópia e de espera.

Memória compartilhada é mais fácil de escrever e mais difícil de escalar; troca de mensagens é o contrário.

CLASSIFICAÇÃO POR LOCALIZAÇÃO FÍSICA DA MEMÓRIA

MEMÓRIA CENTRALIZADA

- Mesma distância de todos os processadores
- Pode ser implementada com um ou vários módulos
- Exemplo: SMP

MEMÓRIA DISTRIBUÍDA

- Composta por vários módulos
- Cada módulo está próximo a um processador
- Exemplo: NUMA, clusters

A localização física impacta diretamente a **latência de acesso**: memória centralizada oferece acesso uniforme; distribuída pode ter latências variáveis.

Consequência no código — *first touch*: o sistema operacional aloca cada página no módulo de memória do processador que a **tocar primeiro**, não no que a declarou. Inicializar um vetor com uma thread só e depois processá-lo com todas coloca todos os dados em um único módulo.

CLASSIFICAÇÃO POR TIPO DE ACESSO À MEMÓRIA

Para multiprocessadores, o problema central é **como acessar a memória** e **garantir coerência** entre caches.

MULTIPROCESSADORES

- **UMA** — Uniform Memory Access
- **NUMA** — Non-Uniform Memory Access
 - • NCC-NUMA
 - • CC-NUMA
 - • SC-NUMA (DSM)
- **COMA** — Cache-Only Memory

MULTICOMPUTADORES

- **NORMA** — No Remote Memory Access
- Apenas acesso local
- Comunicação por mensagens

PROBLEMA-CHAVE

Quando múltiplos processadores com cache local acessam os mesmos dados, como garantir que todos veem valores **consistentes**?

→ Coerência de cache

Resolvida em hardware, mas **o custo é seu**: ela opera sobre *linhas de cache*, não sobre variáveis.

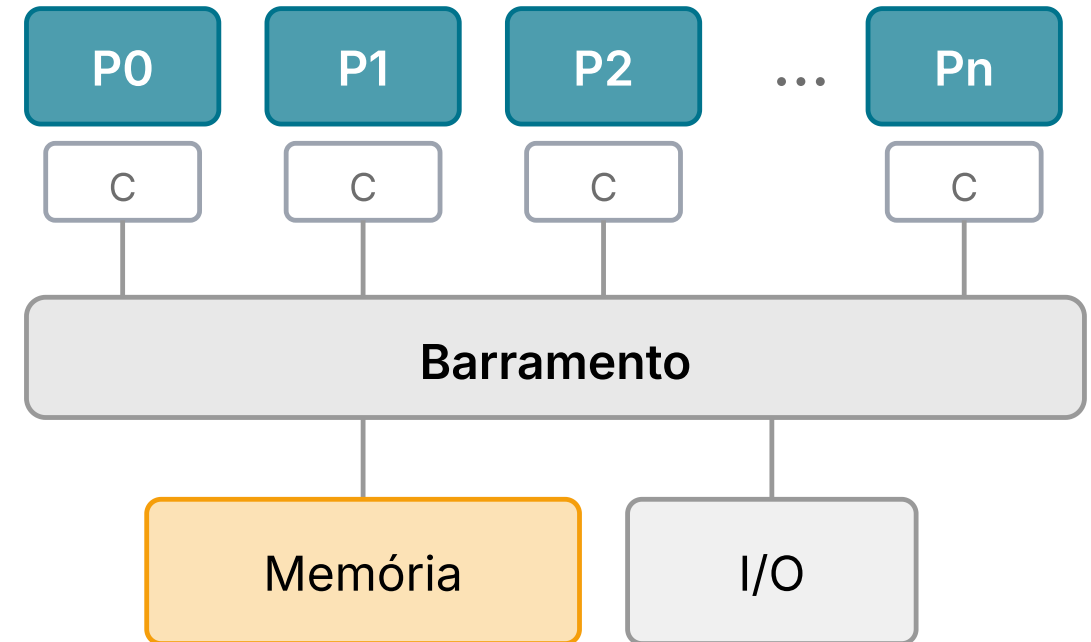
False sharing: duas threads escrevendo em variáveis *diferentes* da mesma linha de cache invalidam a linha uma da outra a cada escrita — programa correto, desempenho pior que o sequencial.

UMA — UNIFORM MEMORY ACCESS

- Memória **centralizada** — mesma distância de todos os processadores
- Custo de acesso **uniforme**
- Coerência de cache tratada em **hardware** (snooping)

Quando um dado compartilhado é alterado por um processador, todas as cópias em outras caches são **invalidadas** ou **atualizadas**.

Cada escrita compartilhada gera **tráfego no barramento** — que é único. Daí o limite prático de algumas dezenas de processadores, e a

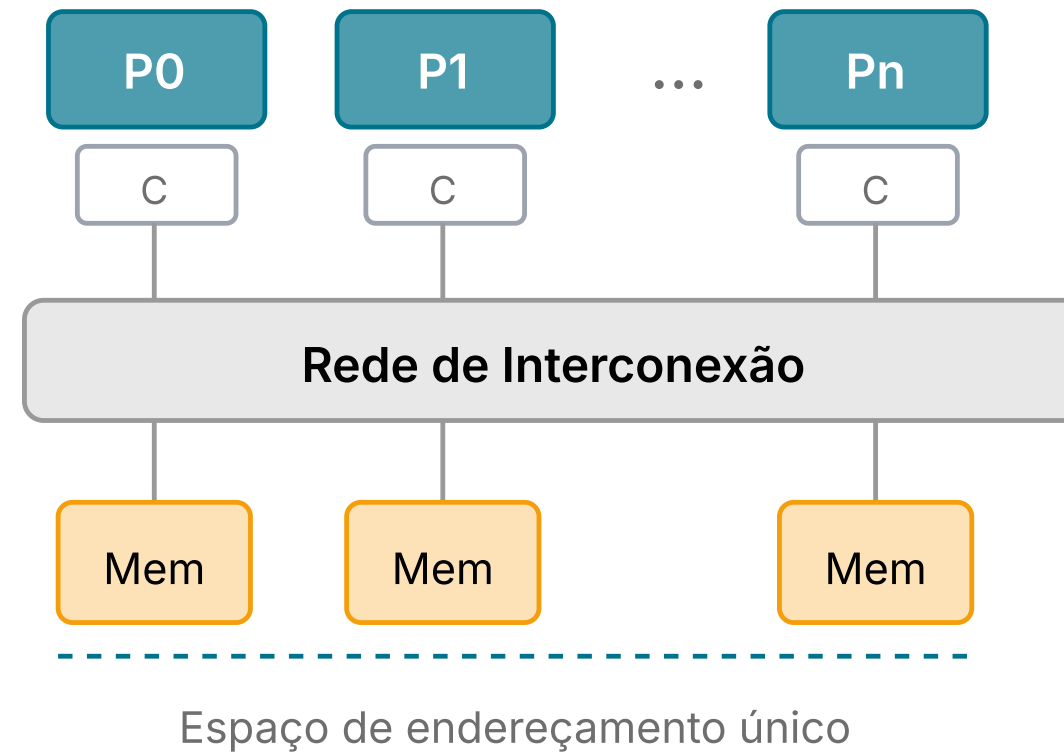


Acesso uniforme — coerência por snooping

NUMA — NON-UNIFORM MEMORY ACCESS

- **Único** espaço de endereçamento
- Memória **distribuída** — distâncias diferentes
- Acesso local é rápido; acesso remoto é mais lento

O custo de acesso depende de **onde** o dado está alocado — acesso remoto custa tipicamente **1,5× a 2×** o local, e a banda é menor.



Ferramentas: numactl -H, lstopo — ver a topologia · numactl --cpunodebind --membind — fixar CPU e memória · OMP_PROC_BIND, OMP_PLACES — afinidade das threads

VARIANTES NUMA E DSM

Variantes diferem no **tratamento da coerência de cache**:

NCC-NUMA

Non Cache-Coherent

Sem garantia de coerência entre caches — o programador é responsável.

CC-NUMA

Cache-Coherent

Coerência em **hardware** — protocolos de diretório.

Modelo dominante em servidores atuais.

SC-NUMA (DSM)

Software-Coherent

Coerência em **software** — Distributed Shared Memory.

Espaço de endereçamento único via abstração de software.

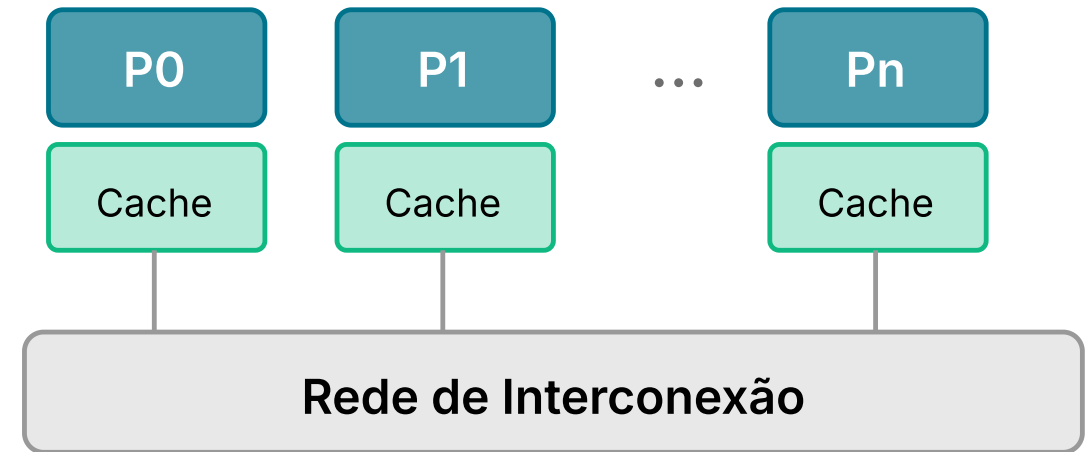
Hoje: a ideia do DSM sobrevive nas linguagens **PGAS** (OpenSHMEM, Coarray Fortran, UPC), que expõem um espaço de endereçamento global sobre um cluster, e na *unified memory* das GPUs, onde CPU e GPU compartilham endereços e o runtime move as páginas.

COMA — CACHE-ONLY MEMORY ARCHITECTURE

- Multiprocessadores com memórias cache de **alta capacidade**
- Memórias locais funcionam como caches — não há "memória principal" fixa
- Dados migram automaticamente para onde são mais usados
- Gerência de caches na MMU

Vantagem: otimização automática de localidade. Desvantagem: complexidade da gerência de migração.

Hoje: não existe COMA comercial, mas a ideia reaparece na *migração automática de páginas* entre nodos NUMA (AutoNUMA no Linux).



Sem memória principal fixa — caches são a memória

NORMA — NO REMOTE MEMORY ACCESS

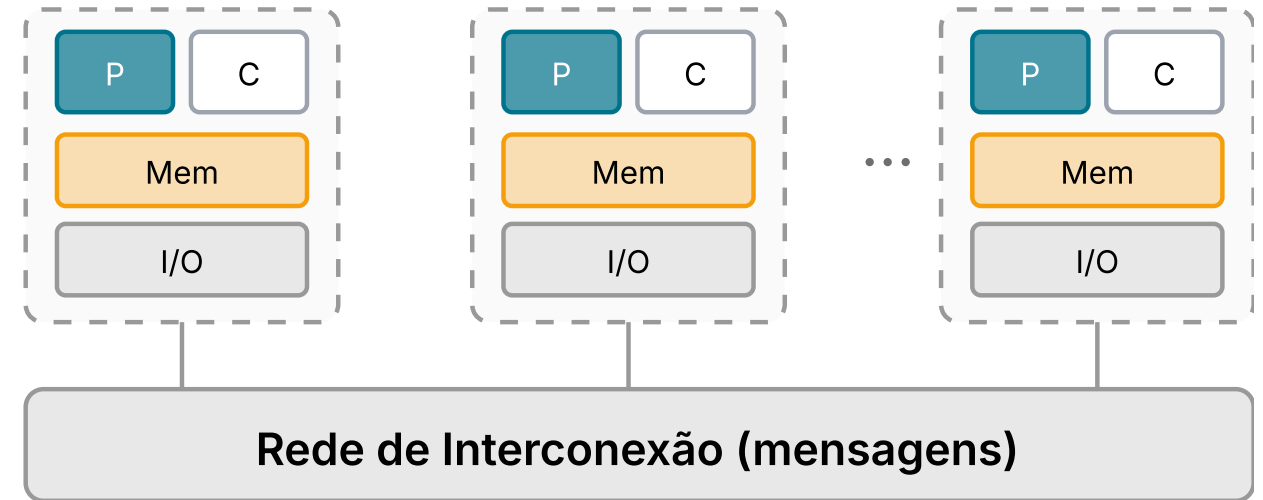
- Apenas acesso **local** à memória
- Comunicação exclusivamente por **troca de mensagens**
- Cada nó é um computador **autônomo** com seu próprio SO

MULTICOMPUTADORES

- **MPP** — Massively Parallel Processors
- **COW** — Cluster of Workstations

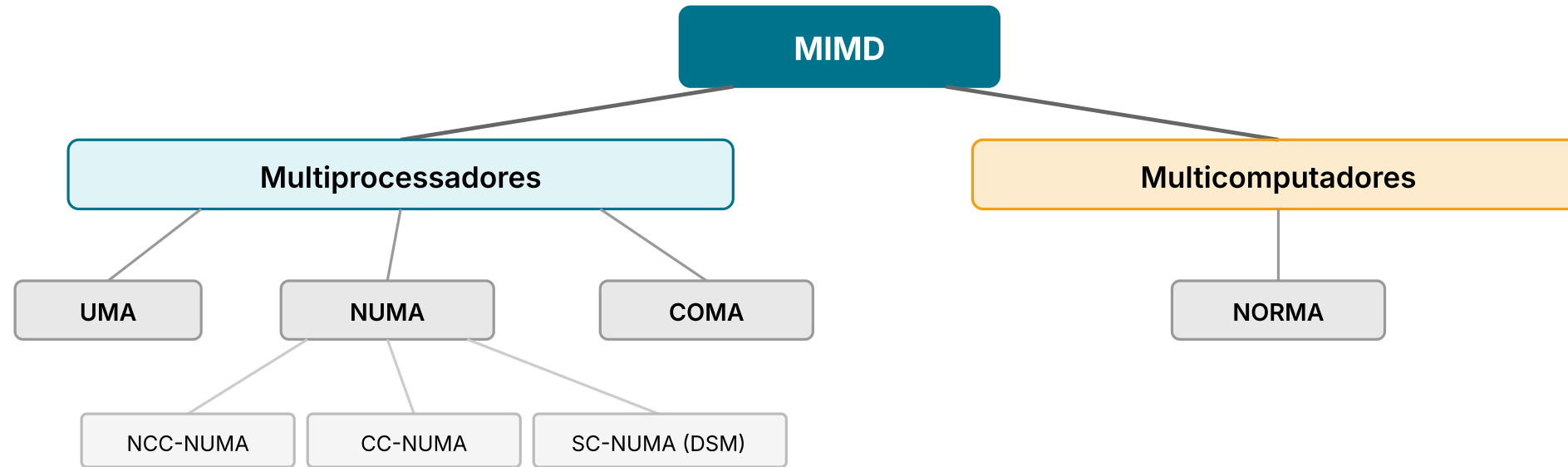
Custo de uma mensagem = latência + tamanho ÷ banda.

A latência é paga *por mensagem*: mil mensagens pequenas custam muito mais que uma grande.



Cada nó: espaço de endereçamento separado

RESUMO: CLASSIFICAÇÃO MIMD



PLATAFORMAS PARA COMPUTAÇÃO PARALELA

Sigla	Plataforma	Classificação		Como se programa
PVP	Processadores vetoriais	SIMD	UMA/NUMA	Vetorização pelo compilador
SMP	Multiprocessadores simétricos	MIMD	UMA/NUMA	OpenMP, pthreads
MPP	Multicomputadores massivamente paralelos	MIMD	NORMA	MPI
NOW	Redes de estações de trabalho	MIMD	NORMA	MPI
COW	Clusters de estações de trabalho	MIMD	NORMA	MPI (+ OpenMP no nodo)
GPU	Aceleradores manycore	SIMD/SIMT		CUDA, OpenACC, OpenMP <i>target</i>

DA ARQUITETURA À FERRAMENTA

Onde está o paralelismo determina como você o explora — e onde isso aparece na disciplina.

Nível	Arquitetura	Modelo	Ferramenta
Dentro do core	Unidades vetoriais (SIMD)	Vetorização	Compilador, #pragma omp simd
Um nodo	Multicore, SMP / NUMA	Threads em memória compartilhada	OpenMP U3
Múltiplos nodos	Cluster, MPP (NORMA)	Processos + mensagens	MPI U4
Acelerador	GPU manycore	Offload SIMT	CUDA, OpenACC
Cluster completo	Nodos multicore em rede	Híbrido	MPI + OpenMP

Os níveis são **cumulativos**: uma aplicação HPC real explora vários ao mesmo tempo.

O QUE LIMITA O DESEMPENHO EM CADA ARQUITETURA

Paralelizar é redistribuir o gargalo — cada classe de máquina tem o seu.

UMA / SMP

Um barramento para todos: cada escrita compartilhada invalida as cópias nas outras caches.

→ Threads a mais param de ajudar; cuidado com *false sharing*.

GPU

Threads que **divergem** executam em série, e mover dados custa mais que calcular.

→ Evite desvios por thread; amortize as transferências.

NUMA

Ler a memória de outro socket custa mais caro — quem decide é **onde a página foi alocada**.

→ *First touch* e afinidade de threads.

CLUSTER (NORMA)

Todo dado que outro nodo precisa **vira mensagem**, e cada mensagem paga latência.

→ Poucas mensagens grandes; carga equilibrada.

O teto, em todos os casos, é a fração sequencial do programa — Lei de Amdahl, na próxima aula.

CONCEITOS FUNDAMENTAIS

Concorrência, paralelismo e distribuição

CONCORRÊNCIA

Concorrência: capacidade de diferentes partes de um programa ou algoritmo de serem executadas fora de ordem ou simultaneamente, sem afetar o resultado final. — *Leslie Lamport*

OBJETIVO PRINCIPAL

Melhor aproveitamento dos recursos e ganho de responsividade.

CARACTERÍSTICAS

- Múltiplas atividades em progresso
- Concorrência física ou lógica (time-sharing)
- Não-determinismo na execução
- Necessidade de sincronização

EXEMPLOS

- Sistema operacional gerenciando processos
- Interface gráfica respondendo a eventos
- Servidor web atendendo múltiplos clientes
- Editor de texto com auto-save e spell-check

PARALELISMO

Sistema paralelo: conjunto de processadores que trabalha cooperativamente para resolver um problema computacional. — Ian Foster

OBJETIVO PRINCIPAL

Ganho de desempenho.

CARACTERÍSTICAS

- Múltiplos processadores/cores
- Execução simultânea real
- Requer decomposição do problema
- Granularidade e balanceamento de carga
- Métricas: speedup, eficiência

EXEMPLOS — OS DESTA DISCIPLINA

- Multiplicação de matrizes
- Algoritmos de ordenação paralela
- Resolução de sistemas de equações
- Processamento de imagens
- Simulações científicas em clusters (HPC)

SISTEMAS DISTRIBUÍDOS

Um **sistema distribuído** é uma coleção de computadores independentes que se apresenta ao usuário como um sistema único e coerente. — Andrew Tanenbaum

OBJETIVO PRINCIPAL

Escalabilidade, tolerância a falhas, compartilhamento de recursos.

CARACTERÍSTICAS

- Componentes autônomos conectados em rede
- Ausência de relógio global
- Falhas parciais e independentes
- Latência de comunicação variável

EXEMPLOS

- Web e serviços em nuvem
- Bancos de dados e sistemas de arquivos distribuídos
- Microsserviços e filas de mensagens
- Redes de sensores e IoT
- **Cluster HPC** — o caso de fronteira

Distribuído no *hardware*, mas com premissas opostas: rede dedicada, nodos fixos, falha aborta o job. Por isso a U4 foca em **desempenho**, não em tolerância a falhas.

COMPARAÇÃO

CONCORRENTE

Estrutura

- Múltiplas atividades em progresso
- Pode ser em 1 processador
- Foco: **corretude**
- Desafio: sincronização

PARALELO

Desempenho

- Execução simultânea real
- Requer múltiplos cores/CPU's
- Foco: **speedup**
- Desafio: decomposição

DISTRIBUÍDO

Escala

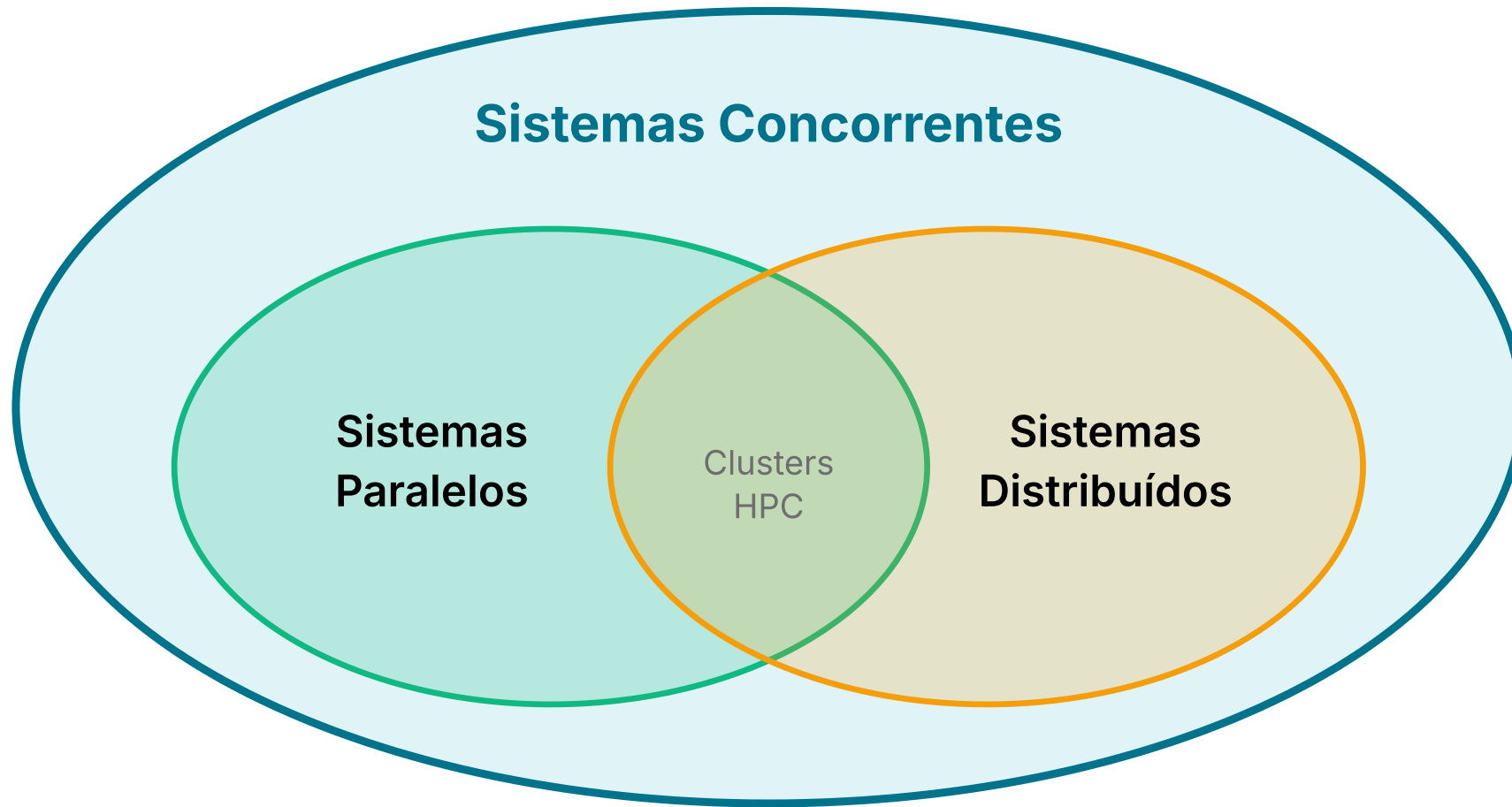
- Computadores independentes em rede
- Falhas parciais possíveis
- Foco: **coordenação**
- Desafio: consistência

"Concurrency is about structure, parallelism is about execution." — Rob Pike

Concorrência é o **conceito unificador** — todo sistema paralelo ou distribuído é concorrente.

COMO SE RELACIONAM?

Paralelismo e distribuição são formas especializadas de concorrência.



A disciplina vive no círculo **paralelo** — e a **Unidade 4**, com MPI em cluster, na interseção.

EXEMPLOS DE CLASSES DE SISTEMAS

Classe		Exemplos	Ferramenta típica
Concorrente		Sistemas operacionais, GUIs, servidores web	Threads, eventos
Concorrente	Paralelo	Multiplicação de matrizes, processamento de imagens, simulação em um nodo	OpenMP , pthreads
Concorrente	Paralelo	Álgebra linear densa, treinamento de redes neurais	CUDA, bibliotecas de GPU
Concorrente	Distribuído	Microserviços, IoT, bancos de dados distribuídos	RPC, filas de mensagens
Concorrente Distribuído	Paralelo	Clusters HPC, simulações de larga escala, treinamento distribuído	MPI , MPI + OpenMP

Na prática, muitos sistemas combinam essas propriedades: um cluster de servidores é **concorrente, paralelo e distribuído** ao mesmo tempo.

DESAFIOS E OPORTUNIDADES

DESAFIOS DE CORRETEUDE

- Comunicação e sincronização
- Não-determinismo: difícil depurar
- Deadlocks, condições de corrida, starvation

DESAFIOS DE DESEMPENHO

- Balanceamento de carga
- Overhead de comunicação e sincronização
- Localidade de dados e *false sharing*
- Limite da fração sequencial

OPORTUNIDADES

- Ganho de desempenho via paralelismo
- Uso eficiente de arquiteturas multicore
- Escalabilidade vertical e horizontal
- Problemas antes inviáveis se tornam tratáveis

Um programa paralelo correto e um programa paralelo rápido são dois problemas distintos — e esta disciplina cobra os dois.

RESUMO

- De von Neumann ao multicore: DMA, caches, pipelines, SMT e Lei de Moore
- Flynn (SISD/SIMD/MIMD) e memória (UMA, NUMA, NORMA) definem o modelo de programação
- Memória compartilhada → **OpenMP**; memória distribuída → **MPI**
- O mesmo algoritmo rende diferente em UMA, NUMA ou cluster — a arquitetura importa no código